C4 Model

Created by Simon Brown, the C4 model is essentially "Google Maps for your code." You start with a zoomed-out view of the world (who uses the system and what external systems it talks to) and progressively zoom in to see the deployable units, the internal structure, and finally the code itself.
Here is your comprehensive guide to understanding and implementing the four levels of the C4 model: **Context, Containers, Components, and Code**.

The Analogy: Zooming In

To understand C4, keep this map analogy in mind:

- **Level 1 (System Context):** The Globe (Big picture, non-technical).
- **Level 2 (Container):** The Country (High-level tech stack).
- **Level 3 (Component):** The City (Internal structure of an application).
- **Level 4 (Code):** The Street View (Classes and interfaces).

Level 1: System Context Diagram

What it interprets:
This is the big picture. It shows your software system as a single box in the center, surrounded by its users (actors) and the external systems it interacts with. It intentionally hides all technical details.
Use Cases:

- Showing product managers, business stakeholders, and non-technical staff how the system fits into the real world.
- Defining the exact boundary and scope of what you are building versus what you are integrating with.

Steps to make it:

1. **Draw your system:** Put your software system in the center as a single box.
2. **Identify Actors:** Add stick figures representing the types of users who interact with your system (e.g., "Customer", "Admin").
3. **Identify External Systems:** Add boxes for any external services your system depends on (e.g., "Stripe Payment Gateway", "Twilio SMS", "Legacy Mainframe").
4. **Connect them:** Draw directional arrows showing who uses what, and label the arrows with simple text explaining the interaction (e.g., "Sends emails using", "Views account balance").

Level 2: Container Diagram

What it interprets:
We zoom into the central "System" box from Level 1. A "Container" in the C4 model represents a separately runnable/deployable unit—**not necessarily a Docker container**. A container could be a Spring Boot backend API, a React Single Page Application, an iOS app, or a PostgreSQL database.

Use Cases:

- Explaining the high-level technical architecture to developers, DevOps, and security teams.
- Showing how responsibilities are distributed across frontends, backends, and databases.

Steps to make it:

1. **Break down the system:** Identify all the runnable units (e.g., Web App, Mobile App, API Gateway, Microservices, Databases).
2. **Specify the tech stack:** Label each box with the technology it uses (e.g., "Java/Spring Boot", "MySQL").
3. **Map the interactions:** Draw arrows between the containers showing how they communicate.
4. **Label protocols:** Add the communication protocols on the arrows (e.g., "JSON/HTTPS", "gRPC", "JDBC").

Level 3: Component Diagram

What it interprets:
We zoom into *one specific Container* from Level 2. This diagram shows the internal building blocks (components) of that application. For a backend API, components usually map to your major controllers, business logic services, and data access repositories.

Use Cases:

- Helping new developers understand the internal structure of a codebase before they start reading actual code.
- Planning a refactoring effort within a specific microservice or monolith.

Steps to make it:

1. **Select a Container:** Choose one box from your Level 2 diagram.
2. **Identify structural blocks:** Map out the high-level internal modules. For example: OrderController, PaymentService, UserRepository.
3. **Define responsibilities:** Briefly describe what each component does inside its box.
4. **Map dependencies:** Draw arrows showing which components call each other, and which components talk to external systems or databases (referencing the boundaries established in Levels 1 and 2).

Level 4: Code Diagram

What it interprets:
We zoom into *one specific Component* from Level 3. This is typically a standard UML class diagram or an Entity-Relationship (ER) diagram showing the actual code-level implementation (classes, interfaces, methods, fields).

Use Cases:

- Deep-dive technical discussions on complex algorithms or design patterns.
- *Note:* Simon Brown (the creator) explicitly recommends **skipping this level** unless absolutely necessary. Code changes too fast, making manual Level 4 diagrams obsolete almost immediately.

Steps to make it:

1. **Do not draw this manually.** 2. Use an IDE plugin (like IntelliJ IDEA's UML generator) to auto-generate class diagrams on demand if a developer needs to see them.

Tools Needed to Create C4 Diagrams

You have two main approaches to creating these diagrams: "Diagrams as Code" (highly recommended for developers) and "Visual Drag-and-Drop."

1. Diagrams as Code (Recommended)

This approach allows you to write text to generate diagrams, which means you can version-control your architecture alongside your code.

- **Structurizr:** Created by Simon Brown himself. It uses a specific DSL (Domain Specific Language) to define your architecture and automatically generates the diagrams. It is the gold standard for C4.
- **PlantUML (with C4-PlantUML):** You can use standard PlantUML text files with a specific C4 library included. It integrates perfectly into Markdown files and GitHub/GitLab.
- **Mermaid.js:** Increasingly popular as it is supported natively by GitHub, Notion, and many Markdown editors. It has recently added support for C4 architecture syntax.

2. Visual Drag-and-Drop

If you prefer drawing manually or collaborating on a whiteboard:

- **Draw.io (diagrams.net):** Free and has built-in C4 model shape libraries.
- **IcePanel:** A fantastic, modern, paid tool specifically built for the C4 model. It links the levels beautifully so you can actually click to "zoom in."
- **Lucidchart / Miro:** Good for collaborative brainstorming, though you will have to manually manage the C4 shapes and connections.

How to Implement C4 on Your Own Today

1. Pick a system you are currently working on.
2. Open **Draw.io** or a **Mermaid.js** live editor.
3. Draw the **Level 1 System Context Diagram** first. Identify who uses your system and what third-party APIs you call.
4. Once that is accurate, draw the **Level 2 Container Diagram**. Map out your frontend, your backend API, and your database.

By mastering just Levels 1 and 2, you will already be communicating system architecture more effectively than the majority of developers.